INFORME Trabajo Práctico

- Base de Datos II

- Unidad 8: Base de Datos para Dispositivos Móviles e Integración

- Integrantes: Ramirez, Deseff, Vargas, Godoy

- **Repositorio del grupo:** https://github.com/FacundoDeseff/Tps_BasedeDatos

Estado del trabajo

- **Consigna 1:** hecha. Están el esquema, las pruebas y la tabla de qué datos viajan. Se volvió a ejecutar `schema_sqlite.sql` + `pruebas_sqlite.sql` en una base temporal: las cuatro tablas son STRICT, las claves foráneas están activas y C1 y C7 dan lo esperado.

- **Consigna 2:** **hecha y probada**. Armamos la instantánea, el semáforo y los cinco casos (C2 a C6). Todos dieron lo que pedía la consigna y PostgreSQL no cambió. La salida completa está en `docs/evidencia_consigna2.txt`. En la revisión de la consigna 3 se corrigieron dos casos borde del semáforo y de la carga del caché (F4 y F5, en 3.c).

- **Consigna 3:** **hecha**. Mapa de los tres motores, plan de mantenimiento y code review con IA. La evidencia de las comprobaciones está en `docs/evidencia_consigna3.txt`.

- **Consigna 4:** completada (spec en Kiro, bitácora, DUIA, README y ensayo de defensa).

1.a Qué datos viajan a la tablet

Durante la feria, la tablet necesita almacenar una copia parcial del sistema. Sin señal, la información que viaja desde la base principal en PostgreSQL hacia la tabla local producto_cache en SQLite y los datos que se dejaron afuera son los siguientes:

| Columna en PostgreSQL | ¿Viaja? | En la tablet | Por qué |

|---|---|---|---|

| `id_producto` | Sí | `id` | Identifica el producto en los borradores y en la sincronización. |

| `nombre` | Sí | `nombre` | Para mostrar el catálogo en pantalla. |

| `precio_lista` (NUMERIC 10,2) | Sí | `precio_centavos` (INTEGER) | Se convierte a centavos enteros para evitar errores de redondeo con decimales. |

| `stock` | Sí | `stock_visto` | Solo orientativo: es el stock que se vio al hacer la copia. |

| `eliminado` | Sí | `eliminado` | Lápida de baja lógica: la tablet debe enterarse de que un producto se dio de baja. |

| *(derivada)* `stock > 0 AND NOT eliminado` | Sí | `disponible` | Permite filtrar el catálogo sin recalcular. |

| *(generada en la tablet)* | — | `fecha_copia` | Indica qué tan vieja es la copia. |

| `id_categoria` | No | — | No hace falta para vender en la feria; se puede sumar si se muestra por categoría. |

| Tabla `usuario` (nombre, apellido) | No | — | Datos personales: no tienen por qué estar en un dispositivo que se puede perder. La tablet solo guarda un alias en `pedido_borrador.usuario`. |

| Tablas `pedido` y `detalle_pedido` | No | — | Son las ventas confirmadas; las decide el servidor. La tablet solo tiene borradores. |

¿Por qué se decidió esto y quién manda?

A la tablet viaja solo lo mínimo para mostrar el catálogo y armar borradores sin conexión; cuanto menos datos sensibles hayan en la tablet, menos riesgo se corre si se extravía. Acá, la fuente de verdad es PostgreSQL ya que el precio, el stock y las ventas los decide el servidor (la copia de la tablet puede estar vieja) por eso lo que la tablet arma es solo una propuesta para que el servidor la valide al sincronizarla con la Base original.

Nota sobre el índice único parcial 'ux_cola_una_pendiente'

Al realizar las pruebas del reenvío, se observó que SQLite prioriza la validación del índica frente a un envío que sigue 'PENDIENTE' (se arroja un error sobre id_local). Una vez que la solicitud se procesa y deja de estar pendiente, actúa la clave primaria de solicitud_uuid, garantizando la idempotencia del sistema.

2. Sincronizar y resolver

2.a La instantánea

Para no tocar la base original, hicimos una copia en PostgreSQL: `foodstore_u8` (sale de `CREATE DATABASE foodstore_u8 TEMPLATE foodstore;`). Ahí armamos la vista `v_snapshot_movil`, que junta lo justo para la feria: id, nombre, precio en centavos, stock, si está disponible, la lápida `eliminado` y la fecha de la copia.

```sql
CREATE VIEW v_snapshot_movil AS
SELECT p.id_producto                    AS id,
    p.nombre                            AS nombre,
    round(p.precio_lista * 100)::bigint  AS precio_centavos,
    p.stock                             AS stock,
    (p.stock > 0 AND NOT p.eliminado)   AS disponible,
    p.eliminado                         AS eliminado,
    now()                               AS fecha_copia
FROM producto p
WHERE p.id_producto IN (1, 3, 4, 5, 6, 7, 8, 10);
```

La feria de este TP trabaja con esos 8 productos, así que la vista se limita a ese catálogo. La copia de la tablet es parcial, como pide la consigna.

Para pasarla a SQLite exportamos el CSV con `\copy ... CSV HEADER` y lo importamos con `.import --csv --skip 1 servidor_snapshot.csv snapshot_servidor`. Ojo: PostgreSQL escribe los booleanos como `t`/`f`, así que `snapshot_servidor` los guarda como texto y `cargar_cache.sql` los pasa a 0/1 al llenar `producto_cache`. Ese llenado lo hacemos con UPSERT para no borrar filas que todavía tienen ítems de borradores apuntándolas.

La copia base nos quedó con 8 productos y fecha **2026-11-04**; el detalle exacto está en la columna `fecha_copia` de `movil/servidor_snapshot.csv`. Las instantáneas de cada caso quedaron en `movil/casos/`.

2.b El semáforo

`semaforo.sql` mira solo las solicitudes que están `PENDIENTE` y las compara con `snapshot_servidor`:

- **RECHAZADO (rojo):** algún producto no existe, está dado de baja (`eliminado = 't'`), no está disponible (`disponible = 'f'`) o no alcanza el stock.
- **CONFLICTO (amarillo):** no hay rojo, pero cambió algún precio.
- **SIMULADO_APROBADO (verde):** todo coincide con la instantánea.

La consulta hace un `LEFT JOIN` de `item_borrador` con `snapshot_servidor` y usa un `CASE` con conteos por solicitud. El `WHERE estado = 'PENDIENTE'` es la clave: si volvés a correr la simulación, no cambia nada. El script no toca PostgreSQL, solo escribe en `cola_sync` de SQLite. Y `SIMULADO_APROBADO` no es una venta: es solo una propuesta que después tiene que validar el servidor.

2.c Los casos

Probamos todo con `python movil/simulador.py`. El simulador arma la base SQLite, importa la instantánea de cada caso y corre `semaforo.sql`. La salida completa está en `docs/evidencia_consigna2.txt`.

En todos los casos el alta del borrador y la solicitud sigue el mismo patrón (cambian los valores):

```sql
INSERT INTO pedido_borrador (id_local, usuario, forma_pago)
VALUES ('<uuid-borrador>', 'cliente_demo', 'EFECTIVO');

INSERT INTO item_borrador (id_local, producto_id, cantidad, precio_centavos_visto)
VALUES ('<uuid-borrador>', <producto>, <cantidad>, <precio>);

INSERT INTO cola_sync (solicitud_uuid, id_local)
VALUES ('<uuid-solicitud>', '<uuid-borrador>');
```

C2 · Precio cambiado. Armamos un borrador del producto 1 (2 unidades a 125945 centavos). Después subimos el precio en el servidor:

```sql
UPDATE producto SET precio_lista = precio_lista + 100.00 WHERE id_producto = 1;
```

Exportamos la instantánea nueva y corrimos el semáforo. Nos dio `CONFLICTO | producto 1: 125945 -> 135945`. El cliente aceptó el precio: actualizamos `item_borrador.precio_centavos_visto` y creamos una solicitud nueva. Ahí nos dio `SIMULADO_APROBADO`. La solicitud vieja queda en `CONFLICTO` para revisión.

C3 · Más unidades que el stock. Armamos un borrador de 192 unidades del producto 3, pero el servidor tiene 142:

```sql
INSERT INTO item_borrador (id_local, producto_id, cantidad, precio_centavos_visto)
VALUES ('c3c3c3c3-0000-4000-8000-000000000003', 3, 192, 391303);
```

Nos dio `RECHAZADO | producto 3: stock 142 < 192`.

C4 · Producto dado de baja. Armamos un borrador del producto 4 y después lo dimos de baja lógica:

```sql
UPDATE producto SET eliminado = true WHERE id_producto = 4;
```

Nos dio `RECHAZADO | producto 4 dado de baja`. Cuando recargamos el catálogo, el producto 4 ya no aparece entre los disponibles, pero la fila sigue en `producto_cache` con `eliminado = 1` (la lápida).

C5 · Sin cambios. Armamos un borrador del producto 5 sin tocar el servidor:

```sql
INSERT INTO item_borrador (id_local, producto_id, cantidad, precio_centavos_visto)
VALUES ('c5c5c5c5-0000-4000-8000-000000000005', 5, 1, 58450);
```

Nos dio `SIMULADO_APROBADO | todo coincide con la instantánea`. Lo corrimos dos veces y quedó igual, porque el semáforo solo mira las solicitudes `PENDIENTE`.

C6 · Reenvío del mismo UUID. Mandamos la misma solicitud con el mismo `solicitud_uuid`:

```sql
INSERT INTO cola_sync (solicitud_uuid, id_local)
VALUES ('a6a6a6a6-0000-4000-8000-000000000006',
    'c6c6c6c6-0000-4000-8000-000000000006');  -- repetido
```

Nos dio el error `UNIQUE constraint failed: cola_sync.solicitud_uuid`; la cola siguió con una sola solicitud y repetir la simulación no reevaluó nada. En un sistema de verdad, la protección no puede quedar en la tablet: el servidor tiene que recibir `solicitud_uuid` como clave de idempotencia y guardarlo en una tabla de solicitudes procesadas (o tener una restricción UNIQUE) dentro de la misma transacción que crea el pedido. Si el UUID ya existe, responde OK sin volver a vender.

Verificación en PostgreSQL

Antes y después de toda la simulación miramos los pedidos y el stock:

```sql
SELECT (SELECT count(*) FROM pedido)                    AS pedidos,
    (SELECT coalesce(sum(stock), 0) FROM producto)      AS stock_total;
```

Nos dio lo mismo en los dos momentos (corrida del 2026-11-04, sobre la copia de ese momento):

Momento	Pedidos	Stock total
Antes de la simulación	200000	4998378
Después de la simulación	200000	4998378

La simulación no cambió ni los pedidos ni el stock total, ni en la copia `foodstore_u8` ni en la original `foodstore`: los totales dependen del estado de la base al momento de correrla, pero la comparación antes/después se mantiene. La tablet nunca cambió el stock ni confirmó ventas. El cambio de precio de C2 y la baja de C4 los hicimos nosotros en la copia del servidor para preparar los casos, no los hizo el semáforo.

La salida completa, con los comandos y los resultados de cada caso, está en `docs/evidencia_consigna2.txt`.

3. Integrar y revisar

3.a Mapa de la capa de datos

Cada motor tiene un papel distinto y un dueño distinto para cada dato. La regla de oro se mantiene: el móvil propone, el servidor dispone.

| Dato | Motor que manda (fuente de verdad) | Copia o derivado | Quién escribe |

|---|---|---|---|

| Catálogo: nombre, `precio_lista`, `stock`, `eliminado` | PostgreSQL (`producto`, `categoria`) | `producto_cache` en SQLite (instantánea parcial); `items[].nombre_producto` y `items[].categoria` en MongoDB (snapshot histórico) | Solo PostgreSQL |

| Clientes | PostgreSQL (`usuario`) | Snapshot `cliente` embebido en cada pedido de MongoDB; en SQLite no viaja (solo el alias `pedido_borrador.usuario`) | Solo PostgreSQL |

| Ventas confirmadas | PostgreSQL (`pedido` + `detalle_pedido`) | Colección `foodstore_nosql.pedidos` en MongoDB (agregado de lectura) | Solo el servidor |

| Borrador en curso | SQLite (`pedido_borrador`, `item_borrador`) | — | La tablet; es una propuesta |

| Estado del envío | SQLite (`cola_sync`) en la simulación; en producción, tabla de solicitudes procesadas en PostgreSQL | — | La tablet encola; el servidor resuelve |

| Precio histórico de una línea | PostgreSQL (`detalle_pedido.subtotal`) | MongoDB lo deriva y lo congela en `items[].precio_unitario_historico` y `items[].subtotal` | El servidor al cerrar la venta |

- **PostgreSQL** es la fuente de verdad del negocio: catálogo, precio, stock y ventas. La tablet nunca cambia esos datos; el semáforo solo escribe en `cola_sync`.

- **MongoDB (Unidad 7)** guarda el pedido como agregado documental (cabecera + cliente + `items[]`). Es una copia de lectura, derivada de PostgreSQL con `03_migracion_pedidos.js` (hoy migra 6 pedidos como fixture); no decide precio, stock ni ventas y se puede regenerar. Su valor es el snapshot histórico: conserva el nombre, la categoría y el precio implícito del momento de la compra.

- **SQLite (tablet)** guarda lo mínimo para vender sin señal: catálogo parcial, borradores y cola de envío. Manda solo sobre el borrador (qué lleva, alias del cliente,

forma de pago) y sobre el estado local del envío. `SIMULADO_APROBADO` no es una venta: es una propuesta que el servidor todavía tiene que validar.

3.b Plan de mantenimiento (agregado al de la Unidad 6)

¿Qué se respalda y cómo se prueba la restauración? Se respalda `foodstore` completo, una vez por día a las 03:00, con `pg_dump -Fc` y escritura en `.parcial` + renombre (TP8). El respaldo incluye la tabla de solicitudes procesadas que garantiza la idempotencia; sin ella, un reenvío después de restaurar podría duplicar una venta. Las tablets no se respaldan: `producto_cache` se reconstruye desde una instantánea y los borradores son descartables porque el servidor no confirmó nada. MongoDB tampoco es crítico: se puede regenerar desde PostgreSQL; si en producción pasara a ser dueño de algún dato, habría que agregar `mongodump` y su prueba de restauración. La restauración se prueba todas las semanas en `foodstore_restore_test` (nunca sobre `foodstore`) con `restore_test.ps1`: conteos contra el respaldo, CHECKs, claves foráneas huérfanas, `pedido.total = SUM(detalle_pedido.subtotal)` y secuencias al día, más una prueba negativa que debe fallar. Se registran RTO y RPO. Para cerrar la capa móvil, después de restaurar se vuelve a crear `v_snapshot_movil` y se repiten C3 y C5 en SQLite: el rojo y el verde tienen que dar lo mismo que antes.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets? El servidor queda atrás y la tablet puede tener datos más nuevos. Al reconectar, el servidor no confía en el `SIMULADO_APROBADO` ni en los precios vistos: descarta la instantánea local, exige una copia completa y vuelve a evaluar cada solicitud pendiente contra el estado restaurado. Si el `solicitud_uuid` ya está en la tabla de procesadas del respaldo, responde OK sin volver a vender; si no está, la solicitud pudo haberse cobrado en la ventana perdida y va a revisión manual, no se aplica automáticamente. La `fecha_copia` de la tablet se compara con la marca del respaldo: si la tablet está adelantada, sus borradores pasan a CONFLICTO o RECHAZADO. La cola local no se borra hasta que el servidor confirme.

¿Cada cuánto y quién revisa CONFLICTO o RECHAZADO? Al cierre de cada feria, el encargado del turno revisa la cola: en CONFLICTO acepta el precio nuevo o

cancela con el cliente; en RECHAZADO avisa al cliente y arma el borrador de nuevo. Una vez por semana, el equipo de datos audita la cola completa y busca patrones (productos que siempre fallan, precios que cambian seguido, stock mal cargado). Cada decisión queda registrada con quién, cuándo y por qué.

3.c Code review con IA

Se pidió a OpenCode (modo plan, solo lectura) que revisara todos los scripts de la cursada (`tp1` a `tp10`), con la condición de que cada hallazgo fuera comprobable en la base. Cada uno se verificó ejecutando SQL de solo lectura sobre `foodstore` o sobre una base SQLite temporal; las consultas están en `docs/verificaciones_consigna3.sql` y la salida completa en `docs/evidencia_consigna3.txt`. Se descartaron los dos falsos positivos que se explican al final. Decisión: **__aceptado__** = hallazgo válido que se documenta como deuda; **__corregido__** = se cambió el código; **__descartado__** = falso positivo.

| # | Hallazgo (archivo) | Decisión | Motivo |

|---|---|---|---|

| F1 · TP3 | `carga_masiva.sql:25-27`: el `LATERAL` no referencia la fila y los 50.000 productos quedaron en una sola categoría | Aceptado | Defecto real de los datos sintéticos; regenerar la carga invalidaría las mediciones y evidencias de TP3/TP5/TP6/TP8. Se documenta como limitación. |

| F2 · TP6 | `tp_desnormalizacion_top_categorias.sql:22,29,44`: la vista "diaria" agrupa por `timestamp` y `fecha = CURRENT_DATE` solo matchea la medianoche | Aceptado | La definición correcta es `ped.fecha::date`. No se recrea la vista para no alterar la evidencia entregada del TP6. |

| F3 · TP4 | `queries_tp4.sql:22,36`: la facturación histórica filtra `pr.eliminado = FALSE` y subestima las ventas | Aceptado | El reporte es histórico y TP5/TP6 no aplican ese filtro. Se documenta la inconsistencia; el filtro debería quitarse o informarse por separado. |

| F7 · TP3 | `carga_masiva.sql:44`: el `subtotal` sintético no se calcula con `precio_lista` | Aceptado | Ya está documentado en TP9: el subtotal es el dato válido

y el precio histórico se deriva de él. Es una característica de los datos, no un error del modelo. |

| F4 · TP10 | `semaforo.sql:27-31`: un borrador sin ítems daba `SIMULADO_APROBADO` | Corregido | Se agregó la rama `NOT EXISTS (item_borrador)`: ahora da `RECHAZADO` con motivo "borrador sin ítems". |

| F5 · TP10 | `cargar_cache.sql:44-47`: una instantánea más angosta dejaba productos viejos como disponibles | Corregido | Se agrega `UPDATE … SET disponible = 0` para los ids que ya no vienen. No se toca `eliminado`: faltar en la copia no es una baja lógica. |

| F11 · TP10 | `importar_snapshot.sql:23-24`: los booleanos del CSV se guardan como `TEXT` | Descartado | Es la tabla de staging del CSV de `psql` (`t`/`f`); `cargar_cache.sql` los pasa a 0/1 y `producto_cache` tiene `CHECK … IN (0,1)`. |

| F12 · TP3 | `equivalencias.sql:5,16`: `COUNT(p.id_producto)` parece un error frente a `COUNT(*)` | Descartado | Con `LEFT JOIN`, `COUNT(column)` es lo correcto: devuelve 0 en categorías vacías. Cambiarlo a `COUNT(*)` sería el bug. |

Cómo se comprobó cada hallazgo desarrollado

- **F1 (TP3).** En `foodstore`: `SELECT id_categoria, count(*) FROM producto GROUP BY 1` devuelve una sola fila, `7 | 50000`; `count(DISTINCT id_categoria) = 1` y `id_categoria = 3` tiene 0 productos. La categoría se eligió una vez y se reutilizó para las 50.000 filas.

- **F2 (TP6).** Agrupando por `ped.fecha` salen 194.094 filas; por `ped.fecha::date`, 368. Para el último día con datos (2026-10-28) hay 230 pedidos por `::date` y 0 por igualdad directa con la fecha. El `WHERE fecha = CURRENT_DATE` compara un timestamp contra la medianoche.

- **F3 (TP4).** La facturación histórica da 1.164.994.959,53 sin el filtro y 1.073.236.662,31 con el filtro: se excluyen 91.758.297,22 (7,88%). Hay 29.432 pedidos vigentes con productos eliminados.

- **F7 (TP3).** De 400.000 detalles, 399.999 tienen `subtotal / cantidad` distinto de `precio_lista`; el ejemplo `id_detalle = 1` da 1.580,35 contra 3.513,87. TP9 ya documenta que el subtotal es el dato válido y el precio histórico se deriva de él.

- **F4 (TP10).** En SQLite, un borrador sin ítems daba `SIMULADO_APROBADO` con motivo "todo coincide con la instantánea"; después del cambio da `RECHAZADO` con motivo "borrador sin ítems: no hay nada para sincronizar". C2 a C6 siguen dando lo mismo.

- **F5 (TP10).** Al pasar de una instantánea con ids 1, 3 y 4 a otra con solo el id 1, antes las tres filas seguían con `disponible = 1`; ahora las ids 3 y 4 quedan en 0. El semáforo ya rechazaba la solicitud (`producto 3 inexistente`), así que el riesgo real era mostrar y dejar armar borradores con un producto que ya no viaja, no aprobar la venta. Esa parte del hallazgo de la IA se corrigió al verificarla.

- **F11 y F12 (descartados).** F11: después de `cargar_cache.sql`, `typeof(disponible)` y `typeof(eliminado)` en `producto_cache` son `integer`; los `TEXT` viven solo en la tabla de staging que recibe el CSV. F12: las dos comprobaciones `EXCEPT` de `equivalencias.sql` devuelven 0 diferencias y las 10 categorías vacías se cuentan como 0, que es justo lo que se busca con `COUNT(column)`.